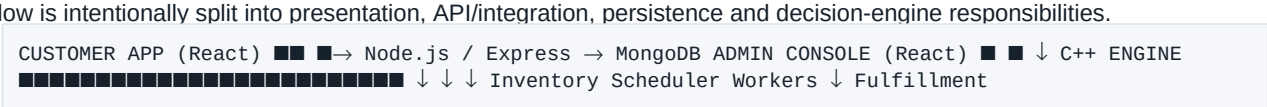
Technical Architecture PRD

System architecture, C++ engine, OOP, DSA, concurrency, APIs and deployment

1. Technical Architecture

QuickFlow is intentionally split into presentation, API/integration, persistence and decision-engine responsibilities.



2. Component Responsibilities

Component	Responsibility
React	Customer storefront, tracking and admin operations UI.
Node.js / Express	REST APIs, authentication, validation, persistence and engine integration.
MongoDB	Persistent operational state and event history.
C++ Engine	Core fulfillment decisions, scheduling, inventory locking and concurrency.
WebSocket / Socket.IO	Real-time state propagation to dashboards.
Docker	Containerized execution of system components.

3. Integration Principle

The customer and admin applications must operate on the same order lifecycle and backend state. The frontend does not communicate directly with the C++ engine; Node.js acts as the API and integration boundary.

Technical Architecture PRD

System architecture, C++ engine, OOP, DSA, concurrency, APIs and deployment

4. C++ Engine — OOP Model

The C++ engine should model the fulfillment domain explicitly rather than placing all logic in a single processor class.

Domain Area	Proposed Classes
Order	Order, Customer, Product, OrderItem
Inventory	Inventory, Warehouse, InventoryManager
Tasks	Task, PickingTask, PackingTask, QualityCheckTask, ShippingTask
Scheduling	Scheduler, FIFOScheduler, PriorityScheduler, DeadlineScheduler, HybridScheduler
Execution	Worker, WorkerManager, ThreadPool
Orchestration	OrderProcessor, WarehouseManager, NotificationManager

5. Design Patterns

Pattern	Application
Strategy	Swap scheduling algorithms without changing fulfillment orchestration.
State	Enforce valid order lifecycle transitions.
Factory	Create task objects based on task type.
Observer	React to order/status changes for notifications, UI and analytics.

6. DSA Requirements

- **Priority Queue:** task ordering for priority/deadline schedulers.
- **HashMap:** fast order, product and worker lookup by ID.
- **Queue:** FIFO work and synchronized task waiting.
- **Graph:** warehouse/location/delivery-network representation.
- **Dijkstra / A*:** proposed route optimization where the modeled network requires shortest-path decisions.

7. Why C++ Is Central

The project’s technical identity comes from using C++ for the parts where correctness and concurrency matter most: inventory reservation, scheduling, worker execution and the thread pool. Node.js remains the communication/API layer.

Technical Architecture PRD

System architecture, C++ engine, OOP, DSA, concurrency, APIs and deployment

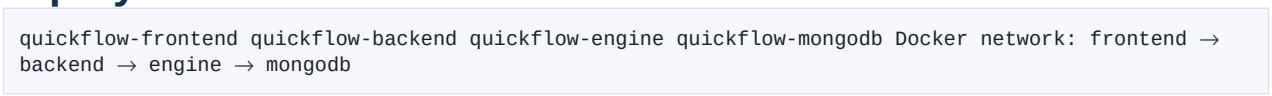
8. Concurrency Architecture

The proposed C++ implementation uses std::thread, std::mutex, std::condition_variable and std::atomic. A mutex/critical section protects shared inventory updates; a condition variable coordinates sleeping worker threads with new work.

9. Backend API Surface

Method	Endpoint	Purpose
POST	/api/auth/login	Authenticate user.
GET	/api/products	List products.
POST	/api/orders	Create an order.
GET	/api/orders/:id	Retrieve order detail.
GET	/api/orders/:id/status	Retrieve current status.
GET	/api/warehouses	Warehouse state.
GET	/api/workers	Worker state.
GET	/api/tasks	Task queue/state.
POST	/api/simulation	Run workload simulation.
GET	/api/analytics	Operational metrics.

10. Data Model

Proposed MongoDB collections include users, products, orders, orderItems, warehouses, inventory, workers, tasks, shipments and events. The order document includes orderId, customerId, items, warehouseId, priority, status, deadline and createdAt.

11. Deployment